Retrieval-Augmented Generation (RAG)

Architecture, Pipeline Dynamics, and System Implementation Manual

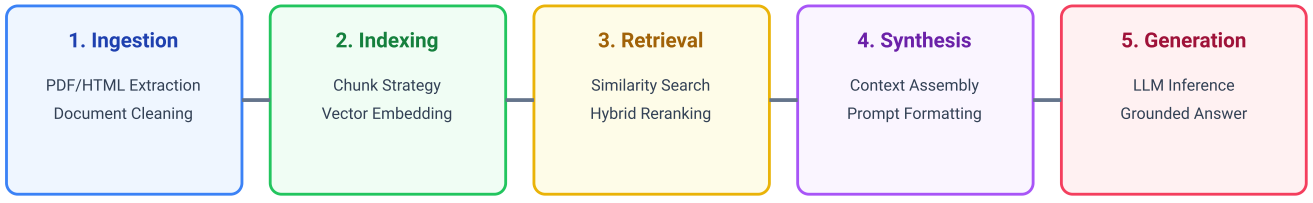
Document Version: 2.4 **Domain:** AI Infrastructure & LLM Systems **Target:** AI Engineers & Architects

1. ARCHITECTURAL OVERVIEW & FOUNDATIONS

Retrieval-Augmented Generation (RAG) is a software pattern that enhances Large Language Models (LLMs) by binding external knowledge repositories directly into the context window at inference time. While foundation models suffer from temporal dynamic cutoffs and static parameters, RAG bridges non-parametric enterprise databases with parametric reasoning models.

85% +	Real-time	100x	ACL / RBAC
HALLUCINATION CUT	DATA FRESHNESS	CHEAPER THAN FT	DATA SECURITY

2. CORE RAG PIPELINE FLOWCHART



3. THE INGESTION & PREPROCESSING SUBSYSTEM

High-performing RAG architectures rely heavily on clean data pipelines. Raw documents (PDFs, Markdown, Web pages, Databases) undergo multi-stage transformations:

- **Parsing & Extraction:** Multi-modal parsers extract text layout, tables, images, and visual metadata preserving hierarchical section boundaries.
- **Text Normalization:** Removing headers, footers, duplicate boilerplate, boilerplate Javascript, and structural noise.
- **Metadata Enriched Chunking:** Injecting parent headers, document titles, access control ACL flags, and creation dates directly into payload headers.

4. CHUNKING STRATEGIES & VECTOR STORAGE

Choosing the right chunking strategy balances semantic cohesion against retrieval granularity. Large chunks provide rich context but lower cosine precision; tiny chunks preserve precision but risk losing sentence relationships.

Fixed-Size Chunking

Splits text every *N* tokens with 10-20% overlap. Computationally fast, but frequently splits key phrases or mathematical context midway.

Semantic & Hierarchy Chunking

Uses embedding distances between sentences or HTML/Markdown headers (`#`, `##`) to split text at natural topic transitions.

5. VECTOR DB COMPARISON MATRIX

Vector Database	Primary Index Type	Filtering Method	Best Use Case
Pinecone	HNSW / Proprietary Graph	Single-stage Metadata	Fully managed cloud enterprise scaling
Qdrant	HNSW with Payload Indexing	Payload-aware filtering	High-performance rust self-hosted/cloud
Milvus	DiskANN / IVF_FLAT / HNSW	Two-stage scalar & vector	Billion-scale massive distributed datasets
pgvector (Postgres)	IVFFlat / HNSW extension	SQL `WHERE` combined queries	Unified relational data & vector storage

6. ADVANCED RETRIEVAL OPTIMIZATION

Naive vector search often fails due to semantic loss or domain-specific language gaps. Production pipelines utilize multi-stage retrieval:

Hybrid Search (Sparse + Dense)

Combines dense vector similarity (cosine distance over embeddings) with sparse keyword matching (BM25 or TF-IDF). The combined scores are normalized using **Reciprocal Rank Fusion (RRF)**:

```
RRF_Score(d) = ∑ ( 1 / (60 + r_dense(d)) ) + ∑ ( 1 / (60 + r_sparse(d)) )
```

- **Cross-Encoder Reranking:** Re-evaluates top-K candidates from initial retrieval using heavy transformer cross-encoders (e.g., Cohere Rerank, BGE-Reranker) to output precise relevance scores.
- **Query Rewriting & HyDE:** Generates hypothetical answer documents (Hypothetical Document Embeddings) to convert questions into hypothetical declarative matches before embedding.

7. CONTEXT ASSEMBLY & PROMPT ENGINEERING

Retrieved context chunks must be packed effectively to counter LLM lost-in-the-middle phenomena (where LLMs prioritize text at the extreme beginning and end of long prompts).

Production Context Prompt Architecture

```
System: You are an expert Enterprise Knowledge Assistant. Answer the question strictly using the provided Context Blocks. If the information is not present, respond "Information unavailable." --- CONTEXT BLOCKS --- [DocID: 104 | Score: 0.94] Revenue grew by 24% in Q3 due to cloud service adoption... [DocID: 209 | Score: 0.88] Operational expenses decreased by 5% following server migration... --- USER QUESTION --- What driven revenue growth in Q3 and how were expenses affected?
```

8. RAG SYSTEM EVALUATION FRAMEWORK (RAGAS)

RAG performance is measured using automated evaluation triads that assess retrieval and generation independently without human gold labels.

Evaluation Metric	Evaluates	Formula Basis / Mechanism
Faithfulness	Generation Quality	Claims in Answer verifiable by Context / Total Claims
Answer Relevance	Generation Quality	Cosine similarity of User Query to generated reverse-questions
Context Precision	Retrieval Quality	Mean Average Precision (mAP) of relevant chunks in top-K
Context Recall	Retrieval Quality	Ground-truth sentences present in retrieved context chunks

9. OPERATIONAL BEST PRACTICES & GOVERNANCE

- **Security & Access Control (ACL):** Implement early filtering at vector store query time so users never retrieve document chunks they lack permission to view.
- **Async Ingestion Queues:** Offload document extraction and embedding generation to continuous background processing queues (e.g., Celery, RabbitMQ) using dead-letter handling.
- **Continuous Cache Invalidation:** Establish dynamic document hashing to automatically evict stale vector records when source database records update.